

Release Notes — v1.0.2

Frontend developer · 2026-07-10 · Partial goods receipt & GRN history

Release date: 2026-07-10 **Backend version:** 1.0.2 **Audience:** Frontend developer

This release adds **partial goods receipt** to Procurement: a purchase order can be received in **several deliveries**, each recorded as an auditable **Goods Receipt Note (GRN)**. This document is self-contained — you can build the whole feature from it.

Live once **backend v1.0.2 is deployed** (needs a DB migration). Only the goods-receipt flow changes — no other module is affected.

At a glance

#	Change	What the frontend does
1	Partial goods receipt — receive a PO in multiple deliveries	Send "arrived now" <code>received_qty</code> (it accumulates); keep receiving until full
2	GRN history per PO	Render a deliveries list from <code>GET /api/purchase-orders/:id/receipts</code>

Endpoints

Method	Path	Role	Purpose
POST	<code>/api/purchase-orders/:id/receive</code>	Purchase, Stores	Record one delivery (GRN)
GET	<code>/api/purchase-orders/:id/receipts</code>	any authenticated	GRN history for the PO

All responses use the standard envelope: success → `{ "success": true, "message": "...", "data": ... }`, error → `{ "success": false, "error": "..." }`.

1. Record a delivery — `POST /api/purchase-orders/:id/receive`

The body shape is unchanged, but `received_qty` now means the quantity that arrived in **THIS delivery** (a delta). It **accumulates** onto the PO line.

Request

```
{
  "warehouse_id": "018f-...",           // required – where accepted stock is booked
  "note": "Courier LR-88213, first lot", // optional, per delivery
  "lines": [
    {
      "po_line_id": "018f-...",           // a line of THIS PO
      "received_qty": 6,                  // arrived in this delivery (delta, not cumulative)
      "rejected_qty": 1,                  // optional, default 0 – counted, never enters stock
      "batch": "LOT-2026-07"             // optional
    }
  ]
}
```

Rules the backend enforces (surface these in the UI):

- The PO must be `Open` or `Partially Received` to receive.
- `received_qty` accumulates; **cumulative received may not exceed the ordered qty** → `400`.
- `rejected_qty` may not exceed this delivery's `received_qty` → `400`.
- At least one line must have `received_qty > 0` → else `400`.
- Accepted qty booked to stock = `received_qty - rejected_qty`.

Success response (`200`)

`data` is the updated PO **with its lines** (note the accumulated `received_qty` and the recomputed `received_pct` / `status`):

```

{
  "success": true,
  "message": "Goods receipt recorded",
  "data": {
    "id": "018f-...", "number": "PO-0007", "supplier_id": "018f-...",
    "status": "Partially Received",    // → "Received" when every line is full
    "received_pct": "60.00",
    "lines": [
      {
        "id": "018f-...", "part_id": "018f-...", "part_number": "MB-VA-15-3040",
        "quantity": "10.000",          // ordered
        "received_qty": "6.000",       // cumulative received so far
        "unit_price": "1199.00", "uom": "Nos"
      }
    ]
  }
}

```

Error responses (`400` , envelope `{ success:false, error }`)

Situation	<code>error</code> message
Over-receipt	<code>received_qty</code> for MB-VA-15-3040 exceeds the remaining quantity (4)
Rejected > received	<code>rejected_qty</code> for MB-VA-15-3040 exceeds received quantity
Nothing received	At least one line must have a <code>received_qty</code> greater than zero
Wrong PO state	PO must be Open to receive goods (currently "Received")
Line not on PO	<code>po_line_id</code> ... does not belong to this PO

UI work

- Label the receive field **"Received now"** (this delivery) — **not** a running total. If your current UI sends the cumulative figure on a second receipt, change it to send only the newly-arrived quantity.
- Compute and show **remaining per line** = `quantity - received_qty` ; pre-fill "Received now" with the remaining, and block input above it.
- Keep the **Receive** action enabled while `status` is `Open` or `Partially Received` ; hide/disable it once `Received` .
- Show the over-receipt/other `400` messages inline on the offending line.

2. Delivery history — `GET /api/purchase-orders/:id/receipts`

Every `/receive` call creates one **GRN**. This returns them, newest first.

Success response (`200`)

```
{
  "success": true,
  "message": "Goods receipts retrieved",
  "data": [
    {
      "id": "018f-...",
      "grn_number": "GRN-0002",
      "po_id": "018f-...",
      "warehouse_id": "018f-...",
      "received_by": "018f-...",
      "received_by_name": "Raj Patel",
      "received_by_initials": "RP",
      "received_by_hue": 152,
      "note": "Balance lot",
      "received_at": "2026-08-01T09:12:00.000Z",
      "lines": [
        {
          "id": "018f-...", "receipt_id": "018f-...", "po_line_id": "018f-...",
          "part_id": "018f-...", "part_number": "MB-VA-15-3040",
          "received_qty": "4.000", "rejected_qty": "0.000",
          "accepted_qty": "4.000", "batch": ""
        }
      ]
    }
  ]
}
```

GRN fields

Header

Field	Type	Use
<code>grn_number</code>	string	Receipt no. (e.g. <code>GRN-0002</code>)
<code>received_at</code>	ISO datetime	Delivery timestamp
<code>received_by</code> + <code>received_by_name/initials/hue</code>	ids + display	Who received it (render name/avatar directly)
<code>note</code>	string	Per-delivery note (courier/LR/remarks)
<code>lines</code>	array	Per-line detail (below)

Line

Field	Type	Use
<code>part_number</code>	string	Material code
<code>received_qty</code>	string	Arrived in this delivery
<code>rejected_qty</code>	string	Rejected on inspection
<code>accepted_qty</code>	string	Entered stock (<code>received</code> – <code>rejected</code>)
<code>batch</code>	string	Lot/batch (may be empty)

Numeric fields come back as **strings** (Postgres numeric) — parse before arithmetic.

UI work

- Add a **"Receipts / Deliveries"** panel on the PO detail page.
- One row per GRN: `grn_number` , `received_at` , `received_by_name` , `note` , and the per-line received/accepted/rejected + batch. A timeline reads well.

Worked example (10 ordered, two deliveries)

1. **Receive 6** → `POST /receive { lines:[{ po_line_id, received_qty: 6, rejected_qty: 1 } ] }`
→ line `received_qty` = `6` , stock **+5** (6 – 1 rejected), PO `Partially Received` , **GRN-0001**.

2. **Receive 4** → `POST /receive { lines:[{ po_line_id, received_qty: 4 }] }` → line `received_qty = 10`, stock **+4**, PO `Received`, **GRN-0002**.
 3. **Try 5 more** → `400 received_qty for ... exceeds the remaining quantity (0)`.
 4. `GET /:id/receipts` → `[GRN-0002, GRN-0001]`, each with its lines.
-

Migration checklist (frontend)

- ☐ Receive form submits **this delivery's** `received_qty` (delta), not the cumulative total; label it "Received now".
- ☐ Show remaining-per-line (`quantity - received_qty`); block/flag over-receipt and surface the `400`.
- ☐ Keep receiving enabled for `Open` and `Partially Received`; disable at `Received`.
- ☐ Add a GRN/deliveries panel from `GET /api/purchase-orders/:id/receipts`.
- ☐ Parse numeric strings before arithmetic.
- ☐ Ship the UI **together with** the backend v1.0.2 deploy.

The migration adds `goods_receipts` and `goods_receipt_lines` tables (`npm run db:push`); existing data is untouched. Full endpoint contract lives in `FRONTEND_API_GUIDE.md` §7.